

# 6

## DEADLOCKS

Computer systems are full of resources that can be used only by one process at a time. Common examples include printers, cameras, microphones, and slots in the system's internal tables. Having two processes simultaneously writing to the printer leads to gibberish. Having two processes using the same file-system table slot invariably will lead to a corrupted file system. Consequently, all operating systems have the ability to (temporarily) grant a process exclusive access to certain resources.

For many applications, a process needs exclusive access to not one resource, but several. Suppose, for example, two processes each want to scan an object with a 3D scanner and then print the front, top, and side view of the object on a printer. Process *A* requests permission to use the 3D scanner and is granted it. Process *B* is programmed differently and requests the printer first and is also granted it. Now *A* asks for the printer, but the request is suspended until *B* releases it. Unfortunately, instead of releasing the printer, *B* asks for the 3D scanner. At this point both processes are blocked and will remain so forever. This situation is called a **deadlock**.

Deadlocks can also occur across machines. For example, many offices have a local area network with many computers connected to it. Often devices such as scanners, printers, and (in some data centers) tape drives are connected to the network as shared resources, available to any user on any machine. If these devices can be reserved remotely (i.e., from the user's home machine), deadlocks of the same kind can occur as described above. More complicated situations can cause deadlocks involving three, four, or more devices and users.